

Remote Volume Control (RVCW)

Security Assessment Report

Study Case

Architecture · Security · Threat Modeling · Recommendations

Analysis Date: May 2026

Contents

1. General Information	3
1.1 Overview	3
1.2 Version History	3
1.3 Assessment Objective	3
1.4 Scope	3
2. Executive Summary	5
3. Architecture Analysis Report	7
3.1 Module & Package Structure	7
3.2 Key Components	7
3.3 Build Configuration	8
4. Android Lifecycle Explanation	9
4.1 MainActivity Lifecycle.....	9
4.2 WebServerService Lifecycle	9
4.3 BootCompletedReceiver Lifecycle	9
4.4 Threading Model.....	10
5. Ktor Server Analysis.....	11
5.1 Server Configuration.....	11
5.2 Installed Plugins	11
5.3 Routes.....	11
5.4 Authentication Mechanism.....	12
6. Security Review	13
6.1 Authentication & Authorization	13
6.2 Transport Security	13
6.3 Input Validation.....	13
6.4 Android Permissions.....	13
6.5 Exported Components	14
6.6 Data Storage	14
7. Threat Modeling	15
7.1 STRIDE Analysis	15
7.2 Attack Scenarios.....	15
Scenario A — Passive Sniffing.....	15
Scenario B — Default Key	15
Scenario C — Cross-Origin Volume Manipulation.....	15
Scenario D — Denial of Service	15

Scenario E — Backup Credential Leak	16
8. Risk Analysis.....	17
9. Recommendations	18
Priority 1 — Critical (fix before production distribution)	18
R1: Move authKey to HTTP header or POST body	18
R2: Enable HTTPS with a self-signed certificate	18
R3: Remove or rotate the default auth key	18
Priority 2 — High	18
R4: Add rate limiting on auth failures.....	18
R5: Bind only to WiFi interface	18
R6: Exclude authKey from Android backup	18
Priority 3 — Medium.....	19
R7: Sanitize StatusPages responses	19
R8: Add CORS configuration	19
R9: Remove CHANGE_NETWORK_STATE permission	19
R10: Fix BroadcastReceiver registration bug	19
Priority 4 — Low	19
R11: Enable ProGuard/R8 in release builds.....	19
R12: Add auth key strength validation.....	19
R13: Expand test coverage	19
10. Lessons Learned.....	20

1. General Information

Remote Volume Control is an Android application developed as a personal project that enables remote control of an Android device's audio volume through a web interface hosted directly on the device.

The application embeds a Ktor HTTP server that listens for incoming requests from devices connected to the same local network. Authorized users can increase, decrease, mute, or set the device volume through a browser-based interface.

The project was originally developed with a focus on functionality and ease of use in trusted home network environments rather than security hardening.

1.1 Overview

Repository: `github.com/RiannReis/remote-volume-control`

Project: Remote Volume Control (RVCW)

Author: Riann dos Reis Silva

Version: 1.0

1.2 Version History

V0.1: [02/05/2026] - First report version

V0.2: [16/05/2026] - Added architecture details

V0.3: [20/05/2026] - Added security review and threat modeling

V0.4: [24/05/2026] - Added Recommendations

V0.5: [28/05/2026] - Images added

V1.0: [08/06/2026] - Final version

1.3 Assessment Objective

The purpose of this assessment is to evaluate the application's security posture through architectural review, threat modeling, attack surface analysis and manual validation of potential security weaknesses.

The assessment focuses on identifying security risks associated with:

- Authentication mechanisms
- Network communication
- Data exposure
- Service availability
- Android-specific security considerations

1.4 Scope

In Scope

Android application package (APK)

- Embedded Ktor HTTP server
- Authentication mechanism
- Web interface
- Android service components
- Local network communication

Out of Scope

- Android operating system
- Third-party infrastructure
- Physical access attacks
- Rooted device scenarios
- External network attacks

2. Executive Summary

This report presents the results of a security assessment conducted against the Remote Volume Control (RVCW) Android application, a personal project designed to provide LAN-based remote audio control through an embedded Ktor web server.

Originally developed with a focus on functionality and ease of use, the application was later revisited as a practical Application Security (AppSec) case study. The assessment combined architectural review, threat modeling (STRIDE), attack surface analysis, static review, and manual validation of potential security findings.

From a software engineering perspective, the application demonstrates a structured architecture and follows several Android development practices. The codebase exhibits separation of concerns between user interface, background services, and HTTP server components. The implementation makes use of Android foreground services, SharedPreferences persistence, permission handling, localization support, and Kotlin-based modular design.

The architectural review identified a number of characteristics, including:

- Separation between UI, service, and networking layers.
- Android lifecycle management.
- Consistent use of Kotlin and Android platform APIs.
- Proper implementation of volume control functionality through Android AudioManager.
- Structured HTTP route definitions using the Ktor framework.

The assessment also identified several security weaknesses, most of which originate from architectural and design decisions rather than implementation defects.

The following findings were manually validated during testing:

- Cleartext HTTP communication (no TLS).
- Authentication secret transmitted through URL parameters.
- Replay vulnerability caused by indefinitely reusable requests.
- Absence of rate limiting or request throttling mechanisms.
- Information disclosure through exception messages returned by the Ktor server.
- Android backup functionality enabled through allowBackup configuration.

A notable result of the assessment was the successful manual validation of findings initially identified during the review. This reinforced the importance of combining automated analysis with practical testing and evidence-based verification.

The overall security posture of the application is consistent with its original purpose as a personal-use utility intended for operation within trusted home networks. However, several attack scenarios become feasible when an attacker gains access to the same network segment, particularly due to the lack of transport encryption, replay protections, and request throttling controls.

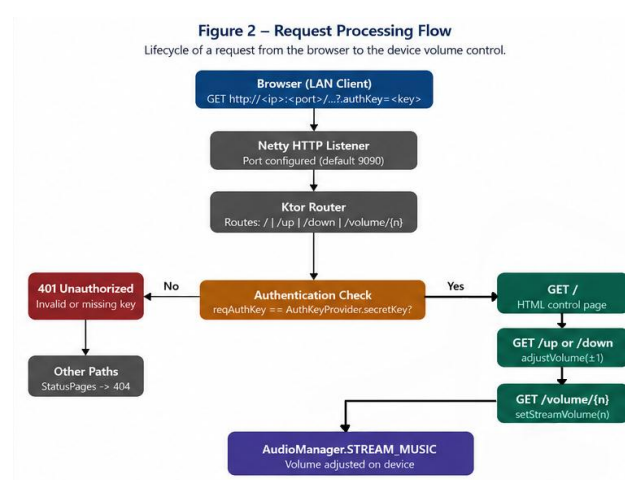
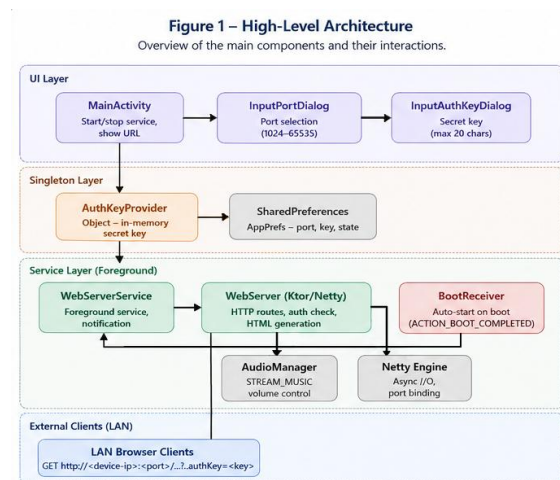
No evidence of remote code execution or privilege escalation vulnerabilities was identified. However, the authentication design was found to be susceptible to credential guessing attacks when weak authentication keys are used, due to the absence of rate limiting or account lockout

mechanisms. Nevertheless, the findings demonstrate several common AppSec issues frequently encountered in internally developed applications and hobby projects.

This assessment highlights the value of applying threat modeling, architecture review, attack surface analysis, and manual validation techniques to existing software projects. Beyond identifying specific findings, the exercise provided a practical opportunity to evaluate a real-world application through a security-focused perspective and document the resulting risks and observations.

3. Architecture Analysis Report

Remote Volume Control (RVCW) is a single-module Android application that exposes an embedded HTTP server on the local area network, enabling any browser on the same WiFi to adjust the device's media volume. The codebase is written in Kotlin targeting Android API 24-34 and uses Ktor 3.0.0 with a Netty engine for HTTP serving.



3.1 Module & Package Structure

The app is organized into one Gradle module (:app) with the following package hierarchy:

- com.riannreis.rvcw — root: AuthKeyProvider singleton
- com.riannreis.rvcw.main — MainActivity (UI entry point)
- com.riannreis.rvcw.dialogs — InputPortDialogFragment, InputAuthKeyDialogFragment
- com.riannreis.rvcw.server — WebServer (Ktor logic), WebServerService (foreground service)
- com.riannreis.rvcw.receiver — BootCompletedReceiver (auto-start)

3.2 Key Components

Component	Type	Responsibility
MainActivity	AppCompatActivity	Primary UI: start/stop service, display URL, dialogs for port/key config.
WebServerService	Foreground Service	Keeps Ktor/Netty server alive; surfaces notification; receives START_STICKY restarts.
WebServer	Plain class	Defines Ktor routes: GET /, /up, /down, /volume/{n}. Validates authKey query param per request. Calls AudioManager.

AuthKeyProvider	Kotlin object (singleton)	In-memory store of the current secret key. Default: "3x@mplE_S3crEt_K3y!"
BootCompletedReceiver	BroadcastReceiver	Listens for ACTION_BOOT_COMPLETED; reads autoStartEnabled from SharedPreferences and starts service if set.
InputPortDialogFragment	DialogFragment	Validates port in range 1024-65535; persists to SharedPreferences "AppPrefs".
InputAuthKeyDialogFragment	DialogFragment	Validates key ≤ 20 chars; persists to SharedPreferences "AppPrefs".

3.3 Build Configuration

- compileSdk / targetSdk: 34 (Android 14).
- minSdk: 24 (Android 7.0 ~97% device coverage).
- Ktor version: 3.0.0 (server-core, server-netty, server-html-builder, server-status-pages).
- Build tools: Gradle 8.7, AGP 8.6.0, Kotlin 1.9.0, JVM target 1.8.
- isMinifyEnabled = false in release, ProGuard/R8 not applied.
- Packaging: META-INF conflicts from Netty excluded via packagingOptions.

4. Android Lifecycle Explanation

4.1 MainActivity Lifecycle

MainActivity follows standard AppCompatActivity callbacks. Key behavior at each stage:

Callback	What RVCW does
onCreate()	Loads saved server state, IP address, port, and auth key from SharedPreferences. Constructs rvcwURL. Requests POST_NOTIFICATIONS permission on Android 13+. Initializes all view references and button listeners.
onStart()	Calls updateActivity() to synchronize UI with isRunning flag. On API 33+ registers serverStateReceiver BroadcastReceiver (RECEIVER_NOT_EXPORTED) to listen for SERVER_STATE_UPDATE broadcasts from the service.
onStop()	Unregisters serverStateReceiver. NOTE: on API < 33 the receiver is never registered (the condition checks API >= O then >= TIRAMISU), so unregisterReceiver() will throw if called without a prior register — a latent bug on API 26–32.
onSaveInstanceState()	Persists isRunning boolean into the Bundle so the UI state survives rotation. The WebServerService itself is unaffected by configuration changes.

4.2 WebServerService Lifecycle

The service is declared as a foreground service with foregroundServiceType="mediaPlayback".

Callback	Behavior
onCreate()	Creates NotificationChannel (API 26+), obtains AudioManager system service, instantiates WebServer.
onStartCommand()	Reads PORT_VALUE and AUTH_KEY from Intent extras. Calls startForeground() with a persistent notification, then webServer.startServer(). Returns START_STICKY — the OS will restart the service after process death, re-delivering the last Intent.
onDestroy()	Calls webServer.stopServer() which calls server.stop(1000, 10000) — a graceful shutdown with 1 s connect and 10 s disconnect grace periods.
onBind()	Returns null — the service is not bound; communication back to the activity uses LocalBroadcasts (SERVER_STATE_UPDATE).

4.3 BootCompletedReceiver Lifecycle

The receiver wakes briefly in onReceive(), reads SharedPreferences, and conditionally starts the foreground service via startForegroundService(). It finishes promptly, as required for BroadcastReceivers (which have a 10-second ANR window). No async work is performed in the receiver itself.

4.4 Threading Model

The application uses three distinct threading contexts:

- Main (UI) thread — All Activity/Fragment callbacks, button click handlers, SharedPreferences reads/writes (via `apply()`, which is asynchronous).
- Ktor/Netty thread pool — Each incoming HTTP request is handled on a Netty worker thread from an internal thread pool. The Ktor routing DSL suspends, but the actual AudioManager calls (`adjustVolume`, `setStreamVolume`) are called from these coroutine contexts without explicit main-thread dispatch.
- IO/Default coroutine dispatcher — `embeddedServer()` is started with `wait=false`, meaning the Netty startup is non-blocking from the Service perspective. The Ktor framework internally dispatches onto its own engine threads.

RISK: AudioManager is generally thread-safe for volume operations, but this is not documented as a thread-safety guarantee. The code relies on an implementation detail.

5. Ktor Server Analysis

5.1 Server Configuration

The embedded server is initialized with Ktor 3.0.0 using the Netty engine:

```
server = embeddedServer(Netty, port = portValue) { ... }
```

Key characteristics:

- Engine: Netty — production-grade, async I/O, NIO-based. Suitable for embedded use.
- Host binding: Default (0.0.0.0) — binds to ALL network interfaces including WiFi, USB tethering, VPN adapters.
- HTTP only — no TLS configuration. All traffic is cleartext.
- `start(wait = false)` — non-blocking startup; exceptions are caught but only logged.
- Stop grace: `stop(1000, 10000)` — 1 s to stop accepting, 10 s for in-flight requests.

5.2 Installed Plugins

Plugin	Configuration
StatusPages	Catches Throwable; responds with 500 + <code>cause.toString()</code> . Leaks internal exception class names and stack details to clients.
HTML Builder	Generates the control page HTML via <code>kotlinx.html DSL</code> . Inline CSS and JavaScript are embedded in the response.

5.3 Routes

Route	Auth	Behavior
GET /	Required	Returns HTML page with current volume slider (STREAM_MUSIC), up/down buttons, and inline JavaScript that re-calls <code>/up</code> , <code>/down</code> , <code>/volume/{n}</code> via XHR.
GET /up	Required	Calls <code>audioManager.adjustVolume(ADJUST_RAISE, FLAG_PLAY_SOUND)</code> . Returns plain text "Volume up".
GET /down	Required	Calls <code>audioManager.adjustVolume(ADJUST_LOWER, FLAG_PLAY_SOUND)</code> . Returns plain text "Volume down".
GET /volume/{n}	Required	Parses path segment as <code>Int</code> ; clamps to <code>[0, maxVolume]</code> ; calls <code>setStreamVolume(STREAM_MUSIC, n, FLAG_PLAY_SOUND)</code> . Returns plain text.
Any other path	—	StatusPages falls through; Ktor default returns 404. Not explicitly handled in code.

5.4 Authentication Mechanism

Authentication is implemented as a simple string equality check on the `authKey` query parameter in every route handler:

```
val reqAuthKey = call.parameters["authKey"]  
if (reqAuthKey == this@WebServer.authKey) { ... }
```

This is query-string authentication — not HTTP Basic Auth, not Bearer tokens, not session cookies. The key is visible in the URL, in server logs, and in browser history. There is no nonce, no replay protection, no rate limiting, and no timing-safe comparison (though string equality in the JVM is effectively constant-time for same-length strings due to JIT optimization differences — this is not guaranteed).

6. Security Review

6.1 Authentication & Authorization

- Query-string secret: The authKey is transmitted in the URL for every request. Any network observer on the LAN can capture it from a single packet. Browser history, proxy logs, and HTTP access logs will also record it.
- No expiry / rotation: The key persists until the user manually changes it. There is no session timeout.
- Default key shipped: The hardcoded default "3x@mpIE_S3crEt_K3y!" is publicly visible in the repository, strings.xml, and BootCompletedReceiver. Users who never change the key are trivially exploitable.
- No rate limiting: An attacker can send unlimited auth attempts. A 20-character ASCII key space is large but a targeted dictionary attack (common patterns, repo default, user-chosen simple keys) has no defense.
- AuthKeyProvider is a Kotlin object: In-memory singleton. The key can be read by any code running in the same process.

6.2 Transport Security

- No TLS: All communication is HTTP. The authKey, volume commands, and HTML page travel in cleartext. Passive LAN sniffing (ARP spoofing, promiscuous mode) trivially captures everything.
- Binding to 0.0.0.0: The server accepts connections on all interfaces. If the device has a USB tethering interface or a VPN tunnel, those are also exposed, not just WiFi.
- No CORS headers: The HTML page includes JavaScript that makes XHR requests. Without CORS, a page on another origin cannot read responses. But the XHR requests (GET /up, /down) are still sent, meaning a malicious web page can trigger volume changes cross-origin if the user visits it while the server is running.

6.3 Input Validation

- /volume/{n}: The volume parameter is parsed with toIntOrNull(). If null, a 400 is returned. Otherwise it is clamped to [0, maxVolume] — correct.
- Port input: Validated to be in [1024, 65535] — correct.
- Auth key input: Only length validation (≤ 20 chars). No character filtering. Special URL characters (e.g. &, =, ?) in the key would break the URL display and potentially the comparison.
- StatusPages: Catching raw Throwable and returning cause.toString() exposes internal exception messages (e.g., java.net.BindException: Address already in use) to HTTP clients — information disclosure.

6.4 Android Permissions

Permission	Assessment
INTERNET	Required for HTTP server. Necessary.

ACCESS_WIFI_STATE / ACCESS_NETWORK_STATE	Used for IP address detection. Necessary.
CHANGE_NETWORK_STATE	Declared but never used in code. Should be removed.
RECEIVE_BOOT_COMPLETED	Required for auto-start feature. Necessary if auto-start is desired.
FOREGROUND_SERVICE	Required. Necessary.
FOREGROUND_SERVICE_MEDIA_PLAYBACK	Specified in manifest and service foregroundServiceType. Necessary for the declared type.
MODIFY_AUDIO_SETTINGS	Required for volume control. Necessary.
POST_NOTIFICATIONS	Required for foreground service notification on API 33+. Necessary.

6.5 Exported Components

- MainActivity — exported=true with LAUNCHER intent filter. Expected and necessary.
- WebServerService — exported=false. Correct. Cannot be started by external apps.
- BootCompletedReceiver — exported=true with BOOT_COMPLETED intent filter. Required for the system to deliver the broadcast. However, this also means any app on the device could potentially send a spoofed BOOT_COMPLETED intent to trigger auto-start (though in practice the system broadcasts are protected). Consider adding a permission check.

6.6 Data Storage

- SharedPreferences ("AppPrefs", MODE_PRIVATE): Stores port, authKey, isRunning, autoStartEnabled. MODE_PRIVATE means only this app can read/write. Correct.
- Backup configuration: backup_rules.xml and data_extraction_rules.xml are present but contain no include/exclude rules. The authKey may be backed up to Google Drive (allowBackup=true in manifest) and restored to a different device or after a factory reset — leaking credentials.

7. Threat Modeling

7.1 STRIDE Analysis

STRIDE	Threat	Scenario	Current Mitigation
Spoofing	Auth key forgery	Attacker sniffs key from HTTP traffic, replays it	Nenhum — transporte em texto não criptografado
Tampering	Volume manipulation	Unauthorized actor sends /up or /down continuously	Verificação de chave de autenticação (ignorável sem TLS)
Repudiation	No audit trail	Cannot determine who changed volume or when	Nenhum
Information Disclosure	Exception leakage	StatusPages returns exception.toString() including class names	Nenhum
Denial of Service	Request flood	Attacker floods /up or /down to max volume and hold	Nenhum — sem limitação de taxa (rate limit)
Elevation of Privilege	MODIFY_AUDIO_SETTINGS abuse	Unauthorized control of device audio settings beyond media volume	Verificação de chave de autenticação

7.2 Attack Scenarios

Scenario A — Passive Sniffing

An attacker on the same WiFi network runs tcpdump or Wireshark. The user opens the URL (e.g., shared via chat or QR code). The attacker captures one HTTP GET request and extracts the authKey from the query string. They now have permanent access until the user changes the key.

Scenario B — Default Key

A user installs the app, enables auto-start, and never configures a custom key. The default key "3x@mplE_S3crEt_K3y!" is publicly documented in the repository README. Any LAN attacker who has seen the repository can immediately access the server.

Scenario C — Cross-Origin Volume Manipulation

A malicious website served from the internet detects the user's local IP (via WebRTC or timing) and sends an XHR to `http://<device-ip>:9090/up?authKey=<guessed>`. Without CORS preflight protection on the server side, the request is sent (though the response body is blocked by CORS). Volume changes still occur.

Scenario D — Denial of Service

An attacker automates curl to send 1000 requests/second to /volume/15?authKey=... (max volume). Netty will queue and process these. The audio system is flooded with volume-set calls. The user cannot lower the volume faster than the attacker raises it.

Scenario E — Backup Credential Leak

Android auto-backup (allowBackup=true, no exclusion rules) backs up the AppPrefs SharedPreferences to Google Drive. The authKey is included. If the user's Google account is compromised, or if they restore the backup to another device, the key is exposed.

8. Risk Analysis

Risk	Likelihood	Impact	Severity	Detail
Cleartext authKey in URL	High	High	Critical	Single packet capture yields permanent access
Default key in public repo	High	High	High	Users who never configure a key are instantly exploitable by anyone who reads the README
No rate limiting on auth	Medium	Medium	High	Dictionary attacks against short/simple user-chosen keys
Binds to all interfaces (0.0.0.0)	Medium	Medium	Medium	Exposes server on USB tethering, VPN, hotspot adapters unintentionally
allowBackup includes authKey	Low	High	Medium	Google account compromise leaks the key
StatusPages leaks exception details	Medium	Low	Low	Internal class names exposed to HTTP clients
CHANGE_NETWORK_STATE unused permission	N/A	Low	Info	Declared but never used; unnecessary attack surface
isMinifyEnabled=false in release	N/A	Low	Info	APK is easy to reverse-engineer; algorithm and key handling are visible
unregisterReceiver() on API 26–32	Low	Medium	Low	Latent crash: onStop() called when onStart() never registered the receiver on those API levels

9. Recommendations

Priority 1 — Critical (fix before production distribution)

R1: Move authKey to HTTP header or POST body

Replace the query-string mechanism with an HTTP Authorization header or a POST body field. This prevents the key from appearing in URLs, browser history, and server access logs.

```
// Ktor server side
val key = call.request.headers["X-Auth-Key"]
// Browser JS side (replace XHR GET)
xhr.setRequestHeader("X-Auth-Key", storedKey);
```

R2: Enable HTTPS with a self-signed certificate

Generate a self-signed TLS certificate at first launch and serve Ktor over HTTPS. The user must accept the browser security warning once, after which all traffic is encrypted. Alternatively, use a local mDNS hostname to enable a valid certificate via Let's Encrypt ACME.

```
import io.ktor.server.engine.sslConnector
sslConnector(keyStore, keyAlias, { passphrase.toCharArray() }, {
    passphrase.toCharArray() }) {
    port = portValue
}
```

R3: Remove or rotate the default auth key

Do not ship a publicly documented default key. On first launch with no saved key, generate a random 16-character key (SecureRandom + Base64) and display it to the user:

```
val random = SecureRandom()
val bytes = ByteArray(12).also { random.nextBytes(it) }
val key = Base64.getUrlEncoder().withoutPadding().encodeToString(bytes)
```

Priority 2 — High

R4: Add rate limiting on auth failures

Track failed auth attempts per source IP using a ConcurrentHashMap with exponential backoff. After 5 failures within 60 seconds, return 429 Too Many Requests and block for 60 seconds.

R5: Bind only to WiFi interface

Instead of binding to 0.0.0.0, detect the WiFi interface IP (already done for display) and bind Netty to that specific IP:

```
embeddedServer(Netty, host = localIp ?: "127.0.0.1", port = portValue) { ... }
```

R6: Exclude authKey from Android backup

Add a specific exclusion in data_extraction_rules.xml to prevent the auth key from being backed up:

```
<cloud-backup>
  <exclude domain="sharedpref" path="AppPrefs.xml"/>
</cloud-backup>
```

</cloud-backup>

Priority 3 — Medium

R7: Sanitize StatusPages responses

Replace `cause.toString()` with a generic error message to prevent exception class names from leaking:

```
call.respondText("Internal server error", status =  
    HttpStatusCode.InternalServerError)
```

R8: Add CORS configuration

Install the Ktor CORS plugin to reject cross-origin requests from non-LAN sources:

```
install(CORS) { allowHost("localhost"); allowSameOrigin = true }
```

R9: Remove CHANGE_NETWORK_STATE permission

This permission is declared in `AndroidManifest.xml` but never used in code. Remove it to reduce the app's declared attack surface.

R10: Fix BroadcastReceiver registration bug

The receiver registration in `onStart()` is guarded by `Build.VERSION.SDK_INT >= O` then `SDK_INT >= TIRAMISU`, meaning it is registered only on API 33+. On API 26–32, `onStop()` calls `unregisterReceiver()` against a never-registered receiver, throwing an exception. Fix:

```
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {  
    registerReceiver(serverStateReceiver, IntentFilter("SERVER_STATE_UPDATE"),  
        RECEIVER_NOT_EXPORTED)  
} else if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {  
    registerReceiver(serverStateReceiver, IntentFilter("SERVER_STATE_UPDATE"))  
}
```

Priority 4 — Low

R11: Enable ProGuard/R8 in release builds

Set `isMinifyEnabled = true` and configure `proguard-rules.pro` to keep Ktor/Netty reflection targets. This makes APK reverse engineering significantly harder.

R12: Add auth key strength validation

Enforce a minimum key complexity: at least 8 characters, mixing letters, digits, and symbols. Display a strength indicator in the dialog.

R13: Expand test coverage

The repository contains only skeleton tests (`ExampleUnitTest`, `ExampleInstrumentedTest`). Add unit tests for WebServer auth logic, volume clamping, port validation, and integration tests for the Ktor routes using Ktor's `TestApplicationEngine`.

10. Lessons Learned

This assessment provided valuable insight into how software architecture decisions directly influence application security.

Although the Remote Volume Control application was originally designed for personal use on trusted home networks, revisiting the project from a security perspective revealed several risks that were not considered during initial development.

The most significant lesson was that security weaknesses are often introduced through architectural design choices rather than implementation bugs. Features such as cleartext HTTP communication, authentication through URL parameters, and the absence of rate limiting functioned correctly from a development perspective but introduced unnecessary attack surface when evaluated from a security standpoint.

The assessment also reinforced the importance of threat modeling. By mapping data flows, trust boundaries, and potential threat actors, it became easier to identify realistic attack scenarios and prioritize risks according to their impact and likelihood.

Another important takeaway was the value of manual validation. Several findings initially identified through static analysis and architecture review required hands-on testing to determine whether they represented practical security concerns. This process helped distinguish theoretical risks from confirmed findings.

Finally, this project demonstrated the benefits of reviewing previously completed software through a different professional lens. What initially started as an Android development project became an opportunity to apply application security concepts, risk assessment methodologies, and structured reporting techniques to a real-world system.

Beyond the technical findings themselves, this assessment demonstrated the value of revisiting previously completed projects with a security-focused mindset. The ability to evaluate, challenge, and reassess past design decisions proved to be as valuable as developing the original application.

This experience strengthened both software development and security analysis skills and highlighted the importance of integrating security considerations throughout the software development lifecycle.